

Relational Algebra

COSC 304 – Introduction to Database Systems



Relational Algebra Query Language

A **query language** allows users to retrieve data from a data model.

Relational algebra is a set of operations for retrieving data in the relational model.

- Like algebra with numbers, relational algebra has operands (relations) and operators.

Every relational operator takes one or more relations as input and produces a relation as output.

A sequence of relational algebra operators is called a **relational algebra expression**.

Relational algebra provides the conceptual foundation for relational query processing. DBMSs translate SQL queries into internal query plans based on these operations.

Relational Algebra Operators

Operator	Symbol	Purpose	SQL Mapping
Selection	σ	Filter rows	WHERE
Projection	Π	Keep only certain columns	SELECT
Cartesian Product	$\times$	Combine two tables in all possible ways	CROSS JOIN
Join	$\bowtie$	Combine two tables based on a condition	JOIN ... ON
Union	$\cup$	Keep rows in either of two tables	UNION
Difference	$-$	Keep rows in first table that are not in second	EXCEPT
Intersection	$\cap$	Keep rows that are in both tables	INTERSECT

Relational algebra operators are fundamental to data processing and occur in other data systems (even non-relational).

Selection Operation

The **selection operation** takes a relation as input and returns a new relation as output that contains tuples that satisfy a condition, called a **predicate**.

- The predicate is similar to a condition in an `if` statement.
- The output therefore has the same degree as the input and cardinality less than or equal to the input.

Selection operation on relation R with predicate F is denoted by $\sigma_F(R)$.

- Predicates may use operands that are attributes or constants/expressions, comparison operators ($<$, $>$, $=$, $\neq$, $\leq$, $\geq$), and logical operators (AND, OR, NOT).

Selection Example

Emp Relation

eno	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

$\sigma_{title = 'EE'}(Emp)$

eno	ename	title	salary
E1	J. Doe	EE	30000
E6	L. Chu	EE	30000

$\sigma_{salary > 35000 \text{ OR } title = 'PR'}(Emp)$

eno	ename	title	salary
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

Selection Question

Question: Given this table and the query:

$\sigma_{salary > 50000 \text{ or } title='PR'}(Emp)$

How many rows are returned?

- A) 0
- B) 1
- C) 2
- D) 3

Emp Relation

<u>eno</u>	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

Selection Question (2)

Question: Given this table and the query:

$\sigma_{salary > 50000 \text{ or } title='PR'}(Emp)$

How many columns are returned?

- A) 0
- B) 2
- C) 3
- D) 4

Emp Relation

<u>eno</u>	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

Projection Operation

The **projection operation** takes a relation as input and returns a relation containing only the specified attributes (columns). Duplicate tuples in the result are eliminated.

- Question: When is projection guaranteed not to eliminate any tuples because of duplicates?

Projection on relation R with output attributes $A_1, \dots, A_m$ is denoted by $\Pi_{A_1, \dots, A_m}(R)$.

- Attributes are output in the order listed.
- Cardinality of $\Pi_{A_1, \dots, A_m}(R)$ may not be the same as R due to duplicate removal.

Projection Example

Emp Relation

<u>eno</u>	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

$\Pi_{eno,ename}(\text{Emp})$

<u>eno</u>	ename
E1	J. Doe
E2	M. Smith
E3	A. Lee
E4	J. Miller
E5	B. Casey
E6	L. Chu
E7	R. Davis
E8	J. Jones

$\Pi_{title}(\text{Emp})$

title
EE
SA
ME
PR

Projection Question

Question: Given this table and the query:

$$\Pi_{title} (Emp)$$

How many rows are returned?

A) 0

B) 2

C) 4

D) 8

Emp Relation

<u>eno</u>	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

Selection and Projection Questions

WorksOn Relation

<u>eno</u>	<u>pno</u>	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

Write the relational algebra expression that:

- 1) Returns all rows with an employee working on project P2.
- 2) Returns all rows with an employee who is working as a manager on a project.
- 3) Returns all rows with an employee working as a manager for more than 40 hours.
- 4) Returns only attributes *resp* and *hours*.
- 5) Returns only *eno*.
- 6) Returns only *pno*.

How many tuples in each output relation?

Union

Union takes two relations R and S and returns all tuples that occur in R , S , or both. Duplicate tuples are eliminated. Syntax: $R \cup S$

R and S must be **union-compatible**:

- 1) Both relations have the same number of attributes.
- 2) Corresponding attributes must have compatible domains/data types.
- Note that attributes do not need to have the same name.
- Result has attribute names of first relation.

Union Example

Emp

<u>eno</u>	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

WorksOn

<u>eno</u>	<u>pno</u>	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

$$\Pi_{eno}(\text{Emp}) \cup \Pi_{eno}(\text{WorksOn})$$

<u>eno</u>
E1
E2
E3
E4
E5
E6
E7
E8

Set Difference

Set difference $R - S$ returns the tuples of R that are not in S .

Note:

- $R - S \neq S - R$
- R and S must be union compatible.

Set Difference Example

Emp Relation

<u>eno</u>	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

WorksOn Relation

<u>eno</u>	<u>pno</u>	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

$$\Pi_{eno}(\text{Emp}) - \Pi_{eno}(\text{WorksOn})$$

<u>eno</u>
E8

Question: What is the meaning of this query?

Question: What is $\Pi_{eno}(\text{WorksOn}) - \Pi_{eno}(\text{Emp})$?

Intersection

Intersection $R \cap S$ returns the tuples that are in both R and S .

- R and S must be union-compatible.

Note that $R \cap S = R - (R - S) = S - (S - R)$.

Intersection Example

Emp

<u>eno</u>	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

WorksOn

<u>eno</u>	<u>pno</u>	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

$$\Pi_{eno}(\text{Emp}) \cap \Pi_{eno}(\text{WorksOn})$$

eno
E1
E2
E3
E4
E5
E6
E7

Set Operations: Union-compatible Question

Question: Two tables have the same number of fields in the same order with the same types, but the names of some fields are different.

True or false: The two tables are union-compatible.

A) true

B) false

Cartesian Product

Cartesian product $R \times S$ pairs every tuple of R with every tuple of S .

If R has degree k_1 and S has degree k_2 , then $R \times S$ has degree $k_1 + k_2$.

If R has cardinality $|R|$ and S has cardinality $|S|$, then $R \times S$ has cardinality $|R| * |S|$.

Cartesian product is also known as **cross product**.

Cartesian Product Example

Emp Relation

<u>eno</u>	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000

Proj Relation

<u>pno</u>	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000

Emp × Proj

<u>eno</u>	ename	title	salary	<u>pno</u>	pname	budget
E1	J. Doe	EE	30000	P1	Instruments	150000
E2	M. Smith	SA	50000	P1	Instruments	150000
E3	A. Lee	ME	40000	P1	Instruments	150000
E4	J. Miller	PR	20000	P1	Instruments	150000
E1	J. Doe	EE	30000	P2	DB Develop	135000
E2	M. Smith	SA	50000	P2	DB Develop	135000
E3	A. Lee	ME	40000	P2	DB Develop	135000
E4	J. Miller	PR	20000	P2	DB Develop	135000
E1	J. Doe	EE	30000	P3	Budget	250000
E2	M. Smith	SA	50000	P3	Budget	250000
E3	A. Lee	ME	40000	P3	Budget	250000
E4	J. Miller	PR	20000	P3	Budget	250000

Cartesian Product Question

Question: R is a relation with 10 rows and 5 columns. S is a relation with 8 rows and 3 columns.

What is the degree and cardinality of the Cartesian product?

- A) degree = 8, cardinality = 80
- B) degree = 80, cardinality = 8
- C) degree = 15, cardinality = 80
- D) degree = 8, cardinality = 18

θ-Join

Theta (θ) join combines tuples from two relations R and S that satisfy a specified condition F . Syntax: $R \bowtie_F S$

Note that $R \bowtie_F S = \sigma_F(R \times S)$.

θ-Join Example

WorksOn Relation

eno	pno	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

Proj Relation

pno	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

WorksOn ⋈_{hours*10000 > budget} Proj

eno	pno	resp	hours	P.pno	pname	budget
E2	P1	Analyst	24	P1	Instruments	150000
E2	P1	Analyst	24	P2	DB Develop	135000
E3	P4	Engineer	48	P1	Instruments	150000
E3	P4	Engineer	48	P2	DB Develop	135000
E3	P4	Engineer	48	P3	Budget	250000
E3	P4	Engineer	48	P4	Maintenance	310000
E4	P2	Programmer	18	P1	Instruments	150000
E4	P2	Programmer	18	P2	DB Develop	135000
E5	P2	Manager	24	P1	Instruments	150000
E5	P2	Manager	24	P2	DB Develop	135000
E6	P4	Manager	48	P1	Instruments	150000
E6	P4	Manager	48	P2	DB Develop	135000
E6	P4	Manager	48	P3	Budget	250000
E6	P4	Manager	48	P4	Maintenance	310000
E7	P3	Engineer	36	P1	Instruments	150000
E7	P3	Engineer	36	P2	DB Develop	135000
E7	P3	Engineer	36	P3	Budget	250000



Types of Joins

The θ -Join is a general join that allows any expression for condition F .

An **equijoin** only contains the equality operator ($=$) in formula F .

- e.g. $\text{WorksOn} \bowtie_{\text{WorksOn.pno} = \text{Proj.pno}} \text{Proj}$

A **natural join** over two relations R and S denoted by $R \bowtie S$ equijoins attributes having the same name in both relations and includes only one copy of each matched attribute.

- Natural join should be used carefully because every same-named attribute is used as a join condition.

Equijoin Example

WorksOn Relation

eno	pno	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

Proj Relation

pno	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

WorksOn ⋈_{WorksOn.pno = Proj.pno} Proj

eno	pno	resp	hours	P.pno	pname	budget
E1	P1	Manager	12	P1	Instruments	150000
E2	P1	Analyst	24	P1	Instruments	150000
E2	P2	Analyst	6	P2	DB Develop	135000
E3	P3	Consultant	10	P3	Budget	250000
E3	P4	Engineer	48	P4	Maintenance	310000
E4	P2	Programmer	18	P2	DB Develop	135000
E5	P2	Manager	24	P2	DB Develop	135000
E6	P4	Manager	48	P4	Maintenance	310000
E7	P3	Engineer	36	P3	Budget	250000

What is the meaning of this join?

Natural join Example

WorksOn Relation

<u>eno</u>	<u>pno</u>	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

WorksOn ⋈ Proj

<u>eno</u>	<u>pno</u>	resp	hours	pname	budget
E1	P1	Manager	12	Instruments	150000
E2	P1	Analyst	24	Instruments	150000
E2	P2	Analyst	6	DB Develop	135000
E3	P3	Consultant	10	Budget	250000
E3	P4	Engineer	48	Maintenance	310000
E4	P2	Programmer	18	DB Develop	135000
E5	P2	Manager	24	DB Develop	135000
E6	P4	Manager	48	Maintenance	310000
E7	P3	Engineer	36	Budget	250000

Proj Relation

<u>pno</u>	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

Natural join is performed by comparing *pno* in both relations and keeping only one copy of it.

Rename Operation

The rename operator ρ gives a relation or its attributes new names. This is useful when the same relation appears more than once in an expression.

Example: $\text{Proj1} \leftarrow \rho_{\text{Proj1}}(\text{Proj})$

$\text{Proj2} \leftarrow \rho_{\text{Proj2}}(\text{Proj})$

Join Practice Questions

Emp Relation

<u>eno</u>	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

Proj Relation

<u>pno</u>	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

WorksOn Relation

<u>eno</u>	<u>pno</u>	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

Compute the following joins (counts only):

- 1) $\text{Emp} \bowtie_{\text{title='EE' and budget} > 400000} \text{Proj}$
- 2) $\text{Emp} \bowtie \text{WorksOn}$
- 3) $\text{Emp} \bowtie \text{WorksOn} \bowtie \text{Proj}$
- 4) $\rho_{\text{Proj1}}(\text{Proj}) \bowtie_{\text{Proj1.budget} > \text{Proj2.budget}} \rho_{\text{Proj2}}(\text{Proj})$

Outer Joins

Outer joins preserve tuples that do not have a matching tuple in the other relation. These unmatched tuples may be called dangling tuples.

There are three types of outer joins:

- 1) **Left outer join** - $R \bowtie\!\!\!\lrcorner S$ - Returns the normal join result plus unmatched tuples from the left relation R , padded with NULL values for attributes from the right relation S .
- 2) **Right outer join** - $R \bowtie\!\!\!\rceil S$ - Returns the normal join result plus unmatched tuples from the right relation S , padded with NULL values for attributes from the left relation R .
- 3) **Full outer join** - $R \bowtie\!\!\!\sqcup S$ - Returns the normal join result plus unmatched tuples from both relations, R and S .

Right Outer Join Example

WorksOn Relation

eno	pno	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

Proj Relation

pno	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

WorksOn $\bowtie_{WorksOn.pno = Proj.pno}$ Proj

eno	pno	resp	hours	P.pno	pname	budget
E1	P1	Manager	12	P1	Instruments	150000
E2	P1	Analyst	24	P1	Instruments	150000
E2	P2	Analyst	6	P2	DB Develop	135000
E3	P3	Consultant	10	P3	Budget	250000
E3	P4	Engineer	48	P4	Maintenance	310000
E4	P2	Programmer	18	P2	DB Develop	135000
E5	P2	Manager	24	P2	DB Develop	135000
E6	P4	Manager	48	P4	Maintenance	310000
E7	P3	Engineer	36	P3	Budget	250000
null	null	null	null	P5	CAD/CAM	500000

Outer Join Question

Question: Given this table and the query:

WorksOn $\bowtie_{WorksOn.pno = Proj.pno}$ Proj

How many rows are returned?

A) 10

B) 9

C) 8

D) 7

WorksOn Relation

eno	pno	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

Proj Relation

pno	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

Semi-Join and Anti-Join

A **semi-join** returns tuples from the first relation that have at least one matching tuple in the second relation. Only attributes from the first relation are returned.

- Semi-joins correspond to SQL queries using `EXISTS` or `IN`.

An **anti-join** returns tuples from the first relation that have no matching tuple in the second relation.

- Anti-joins commonly correspond to SQL queries using `NOT EXISTS`.
- Anti-join is the complement of semi-join: $R \bowtie S = R - R \bowtie S$

Semi-Join Example

WorksOn Relation

<u>eno</u>	<u>pno</u>	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

Proj Relation

<u>pno</u>	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

$\text{Proj} \bowtie_{\text{Proj.pno} = \text{WorksOn.pno}} \text{WorksOn}$

pno	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000

Returns projects that have at least one matching WorksOn tuple. The result contains only attributes from Proj.

Anti-Join Example

WorksOn Relation

<u>eno</u>	<u>pno</u>	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

Proj Relation

<u>pno</u>	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

Proj $\triangleright$ $Proj.pno = WorksOn.pno$ WorksOn

<u>pno</u>	pname	budget
P5	CAD/CAM	500000

Returns projects that have no matching WorksOn tuple.

Combining Operations

Relational algebra operations can be combined in one expression by nesting them:

$$\Pi_{eno,pno,hours}(\sigma_{ename='J. Doe'}(Emp) \bowtie \sigma_{hours>16}(WorksOn))$$

- Return the eno, pno, and hours for employee 'J. Doe' when he has worked on a project for more than 16 hours.

Can also use a temporary relation names for intermediate results.

- The assignment operator $\leftarrow$ can give an intermediate result a name.

```
empdoe  $\leftarrow$   $\sigma_{ename='J. Doe'}(Emp)$ 
wohours  $\leftarrow$   $\sigma_{hours>16}(WorksOn)$ 
empwo  $\leftarrow$  empdoe  $\bowtie$  wohours
result  $\leftarrow$   $\Pi_{eno,pno,hours}(empwo)$ 
```

Operator Precedence

Just like mathematical operators, the relational operators have precedence.

The precedence of operators from highest to lowest is:

- unary operators - σ , Π , ρ
- Cartesian product and joins - $\times$, $\bowtie$
- intersection
- union and set difference

Parentheses can be used to change the order of operations.

It is a good idea to *always* use parentheses to make the intended order of operations explicit.

Core Relational Algebra Operators

The relational operators $\{\sigma, \Pi, \times, \cup, -\}$ form a basic set from which many other relational algebra operators can be derived.

Examples:

Intersection: $R \cap S = R - (R - S)$

Theta join: $R \bowtie_F S = \sigma_F(R \times S)$

Relational Algebra Query Examples

Consider the database schema

- Emp (eno, ename, title, salary)
- Proj (pno, pname, budget)
- WorksOn (eno, pno, resp, hours)

Queries:

- List the names of all employees.
 - $\Pi_{\text{ename}}(\text{Emp})$
- Find the names of projects with budgets over \$100,000.
 - $\Pi_{\text{pname}}(\sigma_{\text{budget} > 100000}(\text{Proj}))$
- Find the names of employees working on project P2.
 - $\Pi_{\text{ename}}(\text{Emp} \bowtie \sigma_{\text{pno} = 'P2'}(\text{WorksOn}))$

Practice Questions

Relational database schema:

branch (bname, address, city, assets)

customer (cname, street, city)

deposit (accnum, cname, bname, balance)

borrow (accnum, cname, bname, amount)

- 1) List the names of all branches of the bank.
- 2) List the names of all deposit customers together with their account numbers.
- 3) Find all cities where at least one customer lives.
- 4) Find all cities with at least one branch.
- 5) Find all cities with at least one branch or customer.
- 6) Find all cities that have a branch but no customers who live in that city.

Practice Questions (2)

branch (bname, address, city, assets)

customer (cname, street, city)

deposit (accnum, cname, bname, balance)

borrow (accnum, cname, bname, amount)

- 1) Find the names of all branches with assets greater than \$2,500,000.
- 2) List the name and cities of all customers who have an account with balance greater than \$2,000.
- 3) List all the cities with at least one customer but without any bank branches.
- 4) Find the name of all the customers who live in a city with no bank branches.

Practice Questions (3)

branch (bname, address, city, assets)

customer (cname, street, city)

deposit (accnum, cname, bname, balance)

borrow (accnum, cname, bname, amount)

- 1) Find all the cities that have both customers and bank branches.
- 2) List the customer name and loan and deposit amounts, who have a loan larger than a deposit account at the same branch.
- 3) Find the name and assets of all branches which have deposit customers living in Vancouver.
- 4) Find all the customers who have both a deposit account and a loan at the branch with name CalgaryCentral.
- 5) Your own?

Other Relational Algebra Operators

Extended relational algebra includes additional operations such as grouping and aggregation (COUNT, SUM, AVG, MIN, MAX).

For example, the basic operators covered cannot answer the query:

What is the total amount of deposits at the Kelowna branch?

We will see how to answer these queries when we study SQL.

Conclusion

Relational algebra provides a set of operations for querying data in the relational model.

Relational algebra operations can:

- Filter tuples — selection (σ)
- Select attributes — projection (π)
- Combine related tuples — join ($\bowtie$)
- Combine all tuple pairs — Cartesian product ($\times$)
- Compare sets of tuples — union ($\cup$), difference ($-$), intersection ($\cap$)

These operators can be combined and nested to express complex queries over multiple relations.

Relational algebra provides the conceptual foundation for relational query processing and maps naturally to SQL.

Objectives

Define relational algebra and explain its relationship to relational query processing.

Apply selection, projection, Cartesian product, union, difference, intersection, and join operations to relations.

Distinguish theta, equi, natural, outer, semi-, and anti-joins.

Combine relational algebra operators to construct expressions for multi-relation queries.

★ Given a relational schema and instance, translate English queries into relational algebra and determine the resulting relation.

Explain how several common relational algebra operations correspond conceptually to SQL operations.



THE UNIVERSITY OF BRITISH COLUMBIA

